



UNIVERSITÉ DE ROUEN NORMANDIE

Master 1 — Génie Informatique et Logiciel

QUORIDOR

Intelligence Artificielle

Implémentation d'un joueur IA par Néma-β

Réalisé par :

- ◆ Mouhous Sidali
- ◆ Younes Khadraoui
- ◆ Pierre Bordeaux
- ◆ Rabah Boudiaf

Unité d'Enseignement : Algorithmes de Recherche et Jeux

Année universitaire : 2025 – 2026

Table des matières

- [1. Introduction et contexte du projet](#)
- [2. Présentation du jeu Quoridor](#)
 - [Règles essentielles](#)
- [3. Architecture logicielle](#)
 - [Développement incrémental](#)
- [4. Algorithme Néma-β et Intelligence Artificielle](#)
 - [4.1 \) Principe du Negamax](#)
 - [4.2 \) Élagage α/β \(Néma-β\)](#)
 - [4.3 \) Niveaux de difficulté](#)
 - [4.4 \) Candidats heuristiques & victoire immédiate](#)
- [5. Heuristique d'évaluation](#)
 - [5.1 \) Formule](#)
 - [5.2 \) Distance BFS](#)
- [6. Interface graphique et expérience utilisateur](#)
 - [Zone de jeu](#)
 - [Panneau latéral](#)
- [7. Analyse des performances](#)
- [8. Difficultés rencontrées et solutions apportées](#)
- [9. Perspectives d'amélioration](#)
- [10. Conclusion](#)
 - [Bilan des objectifs](#)
- [Annexe A : Extraits de code commentés](#)
 - [A.1 \) Néma-β \(ai.py\)](#)
 - [A.2 \) Heuristique \(ai.py\)](#)
 - [A.3 \) BFS distance \(game.py\)](#)

1. Introduction et contexte du projet

Ce rapport présente le travail réalisé dans le cadre de l'unité d'enseignement Algorithmes de Recherche et Jeux du Master 1 Génie Informatique et Logiciel — option ITA de l'Université de Rouen Normandie. L'objectif principal était de concevoir et d'implémenter un joueur artificiel pour le jeu de plateau Quoridor, en utilisant des algorithmes de recherche adversariale vus en cours.

Le projet a été développé en Python 3.11 avec la bibliothèque Pygame 2.6 pour l'interface graphique, et NumPy 1.26 pour la génération de sons procéduraux. Le développement a suivi une approche incrémentale structurée en six versions successives, allant du simple plateau visuel jusqu'à une IA complète avec niveaux de difficulté, animations et effets sonores.

2. Présentation du jeu Quoridor

Quoridor est un jeu de stratégie abstrait pour 2 ou 4 joueurs, créé par Mirko Marchesi et édité par Gigamic. Il se joue sur un plateau de 9×9 cases séparées par des rainures permettant de placer des barrières. Chaque joueur dispose d'un pion et de 10 barrières.



Figure 1 capture du jeu de quoridor

Règles essentielles

- But du jeu : atteindre la rangée adverse en premier. Le joueur 1 (bas) doit atteindre la rangée 1 ; le joueur 2 (haut) doit atteindre la rangée 9.
- Déplacement : À chaque tour, un joueur peut déplacer son pion d'une case orthogonalement. Si les deux pions sont adjacents, un saut par-dessus est possible, ou diagonal si le saut droit est bloqué.
- Placement de barrière : Au lieu de se déplacer, un joueur peut poser une barrière couvrant 2 arêtes consécutives, à condition de ne pas emprisonner un joueur (vérification BFS).
- Fin de partie : Le joueur qui atteint la rangée adverse gagne immédiatement.



Figure 2 Capture de fin de partie

La complexité de Quoridor est estimée à un facteur de branchement moyen de ~60 à 130 selon la phase de jeu.

3. Architecture logicielle

Le projet suit une architecture MVC adaptée à Pygame, avec séparation stricte des responsabilités

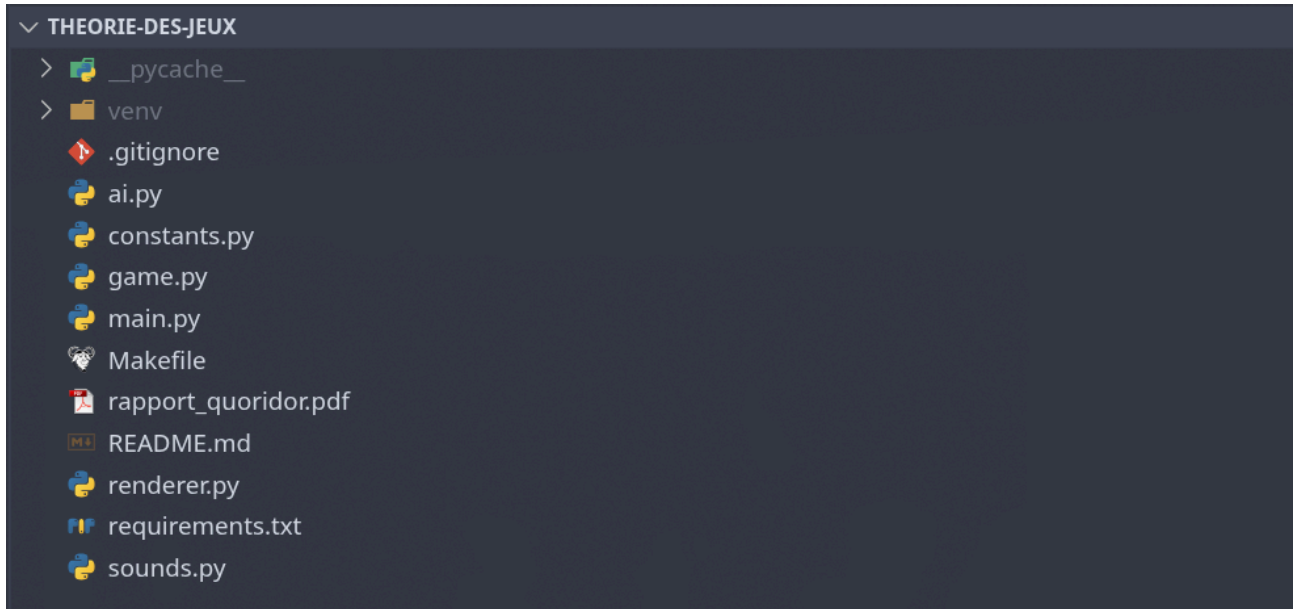


Figure 3 Architecture MVC du jeu

Module	Rôle
constants.py	Constantes globales (tailles, couleurs, polices)
game.py	Logique : positions, coups valides, barrières, BFS, historique
ai.py	IA : Néma-β, heuristique, 4 niveaux de difficulté
renderer.py	Rendu graphique : plateau, pions, barrières, panneau, écrans
sounds.py	Génération procédurale de sons via NumPy + Pygame mixer
main.py	Boucle principale, événements, animations, threading IA

Tableau 1 — Répartition des modules

Développement incrémental

Version	Description
V1	Plateau visuel 9×9
V2	Déplacement des pions, gestion des tours
V3	Barrières H/V, validation BFS, fantôme de prévisualisation
V4	Historique notation officielle, distances BFS dans le panneau
V5	IA Néma-β, sélection mode/niveau/camp, threading
V6	Sons procéduraux, animations pions, écran règles, score session

Tableau 2 — Versions incrémentales

4. Algorithme Néga- β et Intelligence Artificielle

4.1) Principe du Negamax

L'algorithme Negamax est une simplification de Minimax exploitant la propriété de somme nulle des jeux à deux joueurs. Chaque nœud maximise $-\text{score}(\text{enfant})$, unifiant le traitement de tous les nœuds.

```
negamax(état, d) = max{ -negamax(enfant, d-1) }
```

4.2) Élagage α/β (Néga- β)

L'élagage alpha-bêta abandonne les branches qui ne peuvent pas influencer la décision finale. L'élagage coupe dès que $\alpha \geq \beta$. Dans le meilleur cas (ordre de coups optimal), la complexité passe de $O(b^d)$ à $O(b^{d/2})$, doublant la profondeur explorable.

```
def negamax(gs, depth, alpha, beta, ai_player):
    if terminal(gs) or depth == 0:
        return sign * evaluate(gs, ai_player)
    best = -inf
    for action in get_all_moves(gs):
        child = apply(gs, action)
        score = -negamax(child, depth-1, -beta, -alpha, ai_player)
        best = max(best, score)
        alpha = max(alpha, best)
        if alpha >= beta:
            break # élagage  $\beta$ 
    return best
```

4.3) Niveaux de difficulté

Niveau	Profondeur	Comportement
Facile	1	Coup glouton — aucune anticipation
Moyen	2	Anticipe 1 réponse adverse
Difficile	3	Planification à 3 coups, barrières pertinentes
Expert	4	Jeu solide, bloque les chemins adverses

Tableau 3 — Niveaux de difficulté

4.4) Candidats heuristiques & victoire immédiate

Les barrières candidates sont filtrées : rayon 3 autour des pions, maximum 12 par nœud. Cela réduit le branchement de ~ 120 à $\sim 15-20$ coups. Avant Negamax, `best_move()` vérifie systématiquement les coups gagnants immédiats.

5. Heuristique d'évaluation

5.1) Formule

```
score = d_opp × 12 - d_self × 14
        + 200 si d_self == 1 (victoire à 1 coup)
        + 500 si d_self == 0 (victoire immédiate)
        - 150 si d_opp == 1 (danger adverse)
        + (walls_self - walls_opp) × 0.3
```

Le coefficient $14 > 12$ traduit que progresser soi-même est légèrement plus prioritaire que ralentir l'adversaire. Les bonus/malus situationnels orientent fortement l'IA dans les phases critiques.

5.2) Distance BFS

La distance BFS représente le nombre minimal de coups pour atteindre la ligne de but en tenant compte des barrières posées. Elle est admissible (ne surestime pas), garantissant une bonne orientation de la recherche.

Condition	Score
L'IA a gagné	+10 000
L'adversaire a gagné	-10 000
État non terminal	Formule heuristique

Tableau 4 — Cas terminaux

6. Interface graphique et expérience utilisateur

L'interface est organisée en deux zones : le plateau (gauche) et un panneau d'information

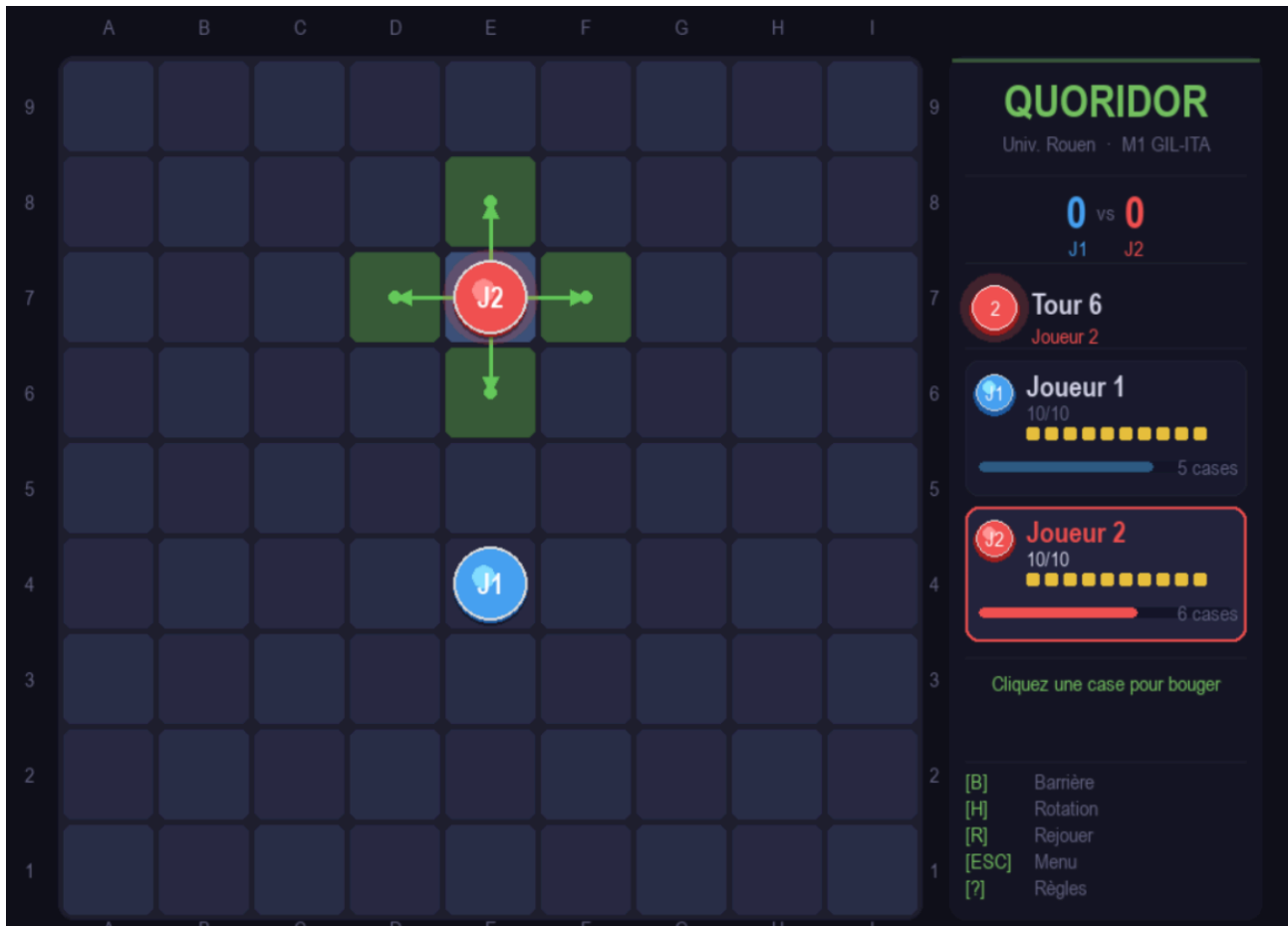


Figure 4 Présentation d'Interface du jeu de quoridor

Zone de jeu

- Pions colorés avec relief (bleu J1, rouge J2)
- Aperçu fantôme des barrières (vert=valide, rouge=invalid)
- Indicateurs de coups valides (cercles semi-transparents)
- Animation des pions : 220 ms, ease-out quadratic

Panneau latéral

- Score de session
- Numéro de tour et joueur actif
- Barrières restantes + distance BFS par joueur
- Raccourcis clavier

7. Analyse des performances

Temps de réflexion mesurés empiriquement. Le threading garde l'UI fluide.

Niveau	Profondeur	Début de partie	Milieu de partie
Facile	1	< 10 ms	< 10 ms
Moyen	2	~50–200 ms	~100–400 ms
Difficile	3	~200 ms – 1 s	~500 ms – 2 s
Expert	4	~1–3 s	~2–6 s

Tableau 5 — Temps de réflexion de l'IA

Sans filtre de candidats, le branchement moyen atteint ~80 coups/nœud. Avec notre filtre (rayon 3 + max 12), il descend à ~15–20 : réduction de 75–80% des nœuds explorés.

8. Difficultés rencontrées et solutions apportées

Problème	Solution apportée
IA ne gagnait pas malgré une case à 1 coup	Vérification préalable des coups gagnants immédiats avant tout appel Negamax.
Flash fantôme de barrière au début du tour humain	Forcer <code>gs.move_mode=True</code> après chaque coup ; calculer <code>wall_hover</code> après tous les événements.
Chevauchement éléments du panneau latéral	Positions y absolues fixes pour chaque section du panneau.
IA lente à profondeur 4	Filtre de candidats + thread daemon séparé pour le calcul IA.
Validation barrières incomplète (cas T et croix)	Réécriture complète de <code>can_place_wall()</code> + vérification BFS systématique.

9. Perspectives d'amélioration

- Tables de transposition (Zobrist hashing) : Mémoriser les positions évaluées pour éviter de recalculer les mêmes sous-arbres.
- Heuristiques incrémentales : Mettre à jour BFS de manière delta plutôt que relancer un calcul complet.
- Algorithme SSS* : Alternative à α/β avec meilleur élagage dans certaines configurations.
- Mode 4 joueurs : Extension via Max-n ou Paranoid pour les parties à 4 pions.
- Apprentissage par renforcement : Réseau neuronal entraîné par auto-jeu pour guider ou remplacer la heuristique.

10. Conclusion

Ce projet a permis d'appliquer concrètement les algorithmes de recherche adversariale à un jeu complexe. La mise en œuvre du Nèga- β avec heuristique BFS calibrée produit un joueur IA compétitif et réactif.

L'approche incrémentale en 6 versions a limité les régressions et facilité le débogage. Le résultat final est une application complète, livrable en un seul dossier Python sans dépendances lourdes.

Bilan des objectifs

Objectif	Détail
Interface graphique	Pygame, animations, sons procéduraux
Règles accessibles	Écran dédié (touche ?)
4 niveaux de difficulté	Facile / Moyen / Difficile / Expert
Nèga- β avec élagage α/β	Implémenté dans ai.py
Heuristique BFS	Distances + bonus/malus situationnels
Python 3.12	Pygame 2.6, NumPy 1.26

Annexe A : Extraits de code commentés

A.1) Nèga- β (ai.py)

```
def negamax(gs, depth, alpha, beta, ai_player):
    if gs.winner is not None or depth == 0:
        sign = 1 if gs.current == ai_player else -1
        return sign * evaluate(gs, ai_player)
    best = -math.inf
    for action in get_all_moves(gs):
        child = copy.deepcopy(gs)
        if action[0] == 'move':
            child.apply_move(action[1], action[2])
        else:
            child.apply_wall(action[1], action[2], action[3])
        score = -negamax(child, depth-1, -beta, -alpha, ai_player)
        if score > best: best = score
        if best > alpha: alpha = best
        if alpha >= beta: break # élagage  $\beta$ 
    return best
```

A.2) Heuristique (ai.py)

```
def evaluate(gs, ai_player):
    if gs.winner == ai_player: return 10_000.0
    if gs.winner == 1 - ai_player: return -10_000.0
    d_self = gs.bfs_distance(ai_player)
    d_opp = gs.bfs_distance(1 - ai_player)
    score = d_opp * 12.0 - d_self * 14.0
```

```
if d_self == 1: score += 200.0
if d_self == 0: score += 500.0
if d_opp == 1: score -= 150.0
score += (gs.walls_left[ai_player] - gs.walls_left[1-ai_player]) * 0.3
return score
```

A.3) BFS distance (game.py)

```
def bfs_distance(self, player):
    start = self.positions[player]
    goal_row = 0 if player == 0 else BOARD_SIZE - 1
    visited = {start}
    queue = deque([(start, 0)])
    while queue:
        (col, row), dist = queue.popleft()
        if row == goal_row: return dist
        for dcol, drow in [(0,-1), (0,1), (-1,0), (1,0)]:
            nc, nr = col+dcol, row+drow
            if (nc,nr) in visited: continue
            if self._blocked(col, row, dcol, drow): continue
            visited.add((nc, nr))
            queue.append((nc, nr), dist+1)
    return 999
```